WASP Software Engineering Assignment

Weijia Shao
Chalmers University of Technology and University of Gothenburg

August 2026

1 Introduction

Modern AI models are increasingly trained using distributed systems in which multiple GPUs across several physical nodes cooperate on a single training job. Frameworks such as PyTorch Distributed and communication libraries such as NCCL coordinate computation and gradient exchange across these workers. The underlying high-performance computing clusters are often multi-tenant: different users may share a physical node while a scheduler such as SLURM isolates their processes and allocated resources. However, this software-level isolation does not necessarily partition shared hardware components such as memory controllers, inter-socket links, PCIe paths, and network interfaces.

My research investigates the security and reliability consequences of this shared infrastructure. In particular, I ask whether an unprivileged co-tenant can deliberately degrade another user's distributed AI training job without exploiting a software vulnerability or obtaining access to its processes, models, or GPUs.

2 Lecture principles

2.1 Principle 1: System Testing

As discussed in the lecture on quality assurance, software testing means systematically experimenting with an executable artifact to evaluate its quality. Testing is therefore not limited to checking individual functions, but can also evaluate integrated and complete systems under realistic operating conditions. This is especially important when software, hardware, requirements, and the surrounding environment interact in complex ways.

My research can be viewed as system-level stress testing of distributed AI training infrastructure. Instead of testing only the model, the PyTorch code, or the communication library, I execute the complete training system under an adversarial but permitted condition: an unprivileged co-tenant aggressively using shared host-memory resources. The relevant quality properties are performance isolation, reliability, and availability rather than model accuracy. A training job may remain functionally correct while still failing as an engineered system if another tenant can increase its step time. My work therefore extends AI testing beyond model outputs to the infrastructure assumptions on which distributed training depends.

2.2 Principle 2: Confirmation Bias and Disconfirming Evidence

The lecture on behavioural software engineering highlighted confirmation bias: engineers and researchers may favour evidence that supports their existing explanations while overlooking contradictory observations. It also emphasized traceability, generating alternative explanations, and explicitly searching for disconfirming evidence as mitigation strategies.

This principle is important in my research because performance attribution in distributed GPU systems is difficult. CPU execution, host memory, GPU kernels, PCIe transfers, inter-socket communication, and network collectives proceed concurrently, so an observed slowdown can have several plausible causes. After seeing a longer AllReduce stage, for example, it would be easy to conclude immediately that the network is the bottleneck while overlooking CPU interference, measurement overhead, workload imbalance, or ordinary cluster variability.

I therefore need to treat each proposed mechanism as a hypothesis that should survive attempts to disprove it. This includes comparing memory and UPI attacks, varying attacker placement and intensity, adding low-intensity and baseline controls, testing workloads with different host-memory and communication profiles, and comparing end-to-end timing with stage-level measurements. Hardware performance counters and communication-library logs provide additional evidence independent of the application timing. The goal is not merely to show that training becomes slower, but to establish a credible causal chain from contention in a particular shared hardware component to the global slowdown of the distributed training job.

3 Guest-lecture principles

3.1 Principle 1: Misuse Cases and Measurable Quality Requirements

The guest lecture on requirements engineering emphasized that broad system goals should be refined into concrete and measurable quality requirements. It also introduced misuse cases, which describe behaviour that a system is supposed to prevent.

This principle applies directly to my research. A general requirement such as "the cluster shall provide secure and reliable multi-tenant AI training" is too vague to evaluate. It can instead be decomposed into measurable properties, such as limiting the slowdown caused by a co-tenant, detecting abnormal contention, and ensuring predictable performance isolation. My attack scenario can also be formulated as a misuse case: a legitimate but adversarial tenant uses only permitted CPU and memory operations to cause another tenant's distributed training job to slow down substantially.

This formulation is important because the attacker does not bypass access control or exploit a software vulnerability. Without an explicit misuse case, the resulting slowdown may be treated merely as performance variability rather than as a failure of security or reliability. The distinction between problem space and solution space is also useful: my research first identifies the missing system guarantee, while mechanisms such as throttling, monitoring, scheduler changes, or hardware partitioning are possible solutions.

3.2 Principle 2: Hidden Gaps in Responsibility and Control

The guest lecture on human aspects of software engineering argued that AI-mediated systems can expose hidden gaps in decisions, responsibilities, and escalation paths. It also presented software engineering as more than code: workflows, controls, roles, accountability, and organizational structures are part of the engineered system.

My research exposes such a gap in multi-tenant AI infrastructure. A scheduler may isolate processes and allocate CPUs and GPUs, but it does not necessarily isolate shared memory controllers, inter-socket links, PCIe paths, or network interfaces. Cluster operators may consider interference a workload issue, framework developers may see it as an infrastructure issue, and users may assume that resource allocation already implies sufficient isolation. As a result, responsibility for preventing hardware-level noisy-neighbour attacks is unclear.

By tracing the effect from a tenant's permitted memory accesses to hardware contention, delayed communication, a straggling worker, and finally a global training slowdown, my work makes this gap visible. The findings suggest that mitigation requires not only technical changes, but also explicit monitoring, performance guarantees, escalation procedures, and clearer responsibility across cluster operators, schedulers, communication libraries, and AI users.

4 Data scientists, software engineers, and AI engineers

4.1 Question 1

The chapters characterise data scientists and software engineers as groups with complementary but different backgrounds, workflows, and success criteria. Data scientists tend to focus on data, statistical methods, model design, experimentation, and predictive accuracy, often through exploratory notebook-based workflows. Software engineers instead focus on delivering and maintaining systems that meet user needs while balancing usability, scalability, security, maintainability, time, and cost. I broadly

agree with this distinction, although it is an oversimplification rather than a strict boundary. It reflects typical training and incentives, not a hierarchy of skills, and many practitioners are better understood as T-shaped: specialised in one area but able to collaborate across both.

4.2 Question 2

The system-wide framing applies directly to my research because the failure I study is invisible from a model-centric perspective. The model architecture, optimiser, and predictions may remain unchanged, while an unprivileged co-tenant creates contention in shared memory controllers, inter-socket links, or network paths. This delays one worker and consequently slows the complete distributed training job. The vulnerability therefore emerges from the interaction among the training algorithm, hardware, communication libraries, and cluster scheduler. It demonstrates that performance, reliability, and security are properties of the complete AI system, not of the model alone.

4.3 Question 3

I expect the roles to remain distinct specialisations, but not isolated skill silos. Data scientists need enough knowledge of deployment, testing, monitoring, and infrastructure to understand how their models behave in a product. Software engineers need enough ML knowledge to reason about model uncertainty, data quality, generalisation, and unavoidable prediction errors. The likely outcome is interdisciplinary teams of T-shaped practitioners rather than everyone becoming equally expert in both fields.

4.4 Question 4

AI Engineering is best understood as a partly distinct discipline at the intersection of software engineering, MLOps, and ML systems work, rather than as a rebranding of any one of them. It inherits software engineering concerns such as requirements, architecture, quality assurance, operations, security, and responsible design, while also addressing ML-specific issues such as data dependence, statistical errors, expensive computation, and changing model behaviour. As the chapters point out, MLOps focuses on making model pipelines easier and more reliable to deploy, update, monitor, and operate, but this pipeline view can remain model-centric if it stops at the interface between the model service and the rest of the product. AI Engineering should therefore take the broader system view advocated in the book, covering interactions among models, software, hardware, users, and the operating environment. In my research, this framing makes hardware-level interference an AI-engineering problem rather than merely an HPC performance issue.

4.5 Question 5

Chapter 2 notes an emerging shift from building individual custom models toward using foundation models and composing them through prompts, prompt chains, and retrieval-augmented generation. Extending this observation, foundation models, coding assistants, RAG systems, prompt pipelines, and agentic workflows make the boundary between data science and software engineering more permeable. Some work moves away from model training and toward composing models, retrieval components, tools, and fallbacks into reliable systems. This increases the importance of software-engineering skills such as architecture, evaluation, monitoring, security, and failure handling, while creating hybrid tasks such as prompt design, retrieval evaluation, and agent orchestration. Coding assistants also allow data scientists to implement software more easily and software engineers to prototype AI features, although neither role can safely ignore the other's expertise. The distinction therefore shifts from "building models" versus "building software" toward collaboration across models, data, prompts, tools, infrastructure, and users.

5 Paper analysis

5.1 Paper 1: Modeling Resilience of Collaborative AI Systems

Core ideas and software-engineering importance. Rimawi et al., 2024 propose a framework for modelling the resilience of Collaborative Artificial Intelligence Systems (CAISs), where an AI component works together with a human to achieve a shared goal. Their main idea is that resilience should not be assessed only through a final accuracy value. Instead, engineers should monitor how system performance evolves before, during, and after a disruptive event. The framework divides execution into states, including a steady state, a disruptive state, a recovered state, and a final state after the source of disruption is removed. It then uses measurable quantities to determine whether and how quickly the system returns to an acceptable level of operation.

The paper demonstrates the framework using a collaborative robot that learns online from human demonstrations to classify objects by colour. The robot normally performs classifications autonomously, but falls back to human assistance when its confidence is too low. The authors introduce the Autonomous Classification Ratio as a measure of system performance and deliberately disrupt the camera input by switching off its supporting lights. They then track the resulting degradation and recovery. This is important to software engineering because it treats resilience as an observable, system-level quality property rather than as an informal expectation. It also connects runtime monitoring with operational decisions: engineers can define an acceptable threshold, detect when the system falls below it, and evaluate the effort and time required for recovery.

Relation to my research. The paper relates closely to my research because both investigate how a local disruptive event propagates into degradation of a larger AI system. In the paper, the disruption changes sensor conditions and reduces the ability of an online-learning robot to operate autonomously. In my work, an unprivileged co-tenant creates contention in shared host hardware, delaying one worker in a distributed training job and consequently slowing all workers.

An important mechanistic asymmetry sharpens this comparison. In the CAIS setting the AI component itself adapts online and eventually closes the disruptive state without external intervention. In synchronous distributed training there is no equivalent self-healing: because the disruption sits in the infrastructure rather than the model, the AllReduce barrier propagates the damage across all workers instead of absorbing it, and recovery depends on operator action rather than on continued learning.

My current evaluation focuses mainly on slowdown ratios, throughput loss, and stage-level latency under attack. The resilience perspective suggests a broader temporal analysis. For example, I could study how rapidly training performance degrades after an attacker starts, whether the system reaches a stable degraded state, and how quickly it recovers after the attacker stops. This would distinguish resistance to disruption from recovery after disruption, and would allow two systems with the same maximum slowdown to be compared more meaningfully if one recovers much faster than the other.

Integration into a larger AI-intensive project. Consider a multi-tenant university platform used to train foundation models, vision models, and graph neural networks on a shared GPU cluster. Its core functionality includes distributed training, job scheduling, resource allocation, communication through NCCL, and continuous collection of hardware and application-level metrics. A resilience manager could apply the paper's state-based model to each training job. Normal throughput would establish a steady-state threshold; a sudden drop caused by contention would start a disruptive state; and restoration of throughput after migration, throttling, or attacker termination would indicate recovery.

My research would fit into this project by supplying both a validated disruption mechanism and the hardware-counter signals needed to attribute it. Hardware performance counters, GPU timing, communication logs, and topology information could explain why the job crossed its resilience threshold. The platform could then use this evidence to distinguish adversarial contention from benign variation and trigger an appropriate response, such as job relocation, resource throttling, or administrator investigation.

Adaptation of my research. To better support the paper's resilience idea, I would extend my experiments from fixed attack intervals to complete disruption-and-recovery timelines. Rather than reporting only baseline and attacked averages, I would measure the onset time, degradation depth,

accumulated lost training progress, recovery time, and any residual performance effect after the attack ends. I would also investigate whether different training frameworks or scheduling policies degrade gracefully or abruptly. In the longer term, this could shift the project from demonstrating an availability attack toward evaluating and designing resilient distributed-training infrastructure.

5.2 Paper 2: Trustworthy Distributed AI Systems: Robustness, Privacy, and Governance

Core ideas and software-engineering importance. Wei and Liu, 2025 present a broad framework for trustworthy distributed AI based on three main pillars: robustness, privacy, and fairness, supported by data and model governance. The paper surveys threats that arise because distributed learning involves multiple participants, communication channels, data sources, and administrative domains. Its robustness taxonomy includes evasion attacks during inference, poisoning and Byzantine attacks during training, and irregular or non-IID data. Its privacy discussion covers attacks such as gradient leakage and inference from model outputs, while its governance discussion emphasizes monitoring, auditing, accountability, policy enforcement, and compliance across the AI lifecycle.

The paper is important to software engineering because it treats trustworthiness as an architectural and lifecycle concern rather than as a property that can be added to a trained model afterwards. The authors argue that trustworthy distributed AI should include built-in safeguards, countermeasures, governance structures, and mechanisms for detecting, preventing, and mitigating misuse. They also stress that robustness, privacy, and fairness interact and therefore cannot be engineered entirely in isolation.

Relation to my research. My research fits most directly under the paper's robustness and security themes, but it also exposes a gap in its taxonomy. The survey mainly discusses model- and data-level disruptions, including poisoning, Byzantine behaviour, adversarial examples, privacy leakage, and irregular data distributions. My work instead studies an infrastructure-level availability attack: a legitimate co-tenant uses permitted memory operations to interfere with shared hardware and slow distributed training without modifying the model, gradients, or training data. Described in the survey's own vocabulary, this is a new row in its threat table: attack target *infrastructure / data-in-use*, attack location *co-tenant on the same physical node*, attack timing *training*, attack effect *denial of service and cost inflation*.

A model can remain statistically correct and private while the system that trains it becomes economically or operationally unusable. In expensive distributed training, severe slowdown may increase cost, delay deployment, waste allocated cluster time, or prevent time-critical model updates. This motivates extending the survey's robustness pillar with an infrastructure-integrity dimension alongside its existing algorithmic categories.

Integration into a larger AI-intensive project. The same multi-tenant AI training platform can illustrate how the paper's ideas would be applied. The platform could maintain a trustworthiness layer covering model integrity, privacy, fairness, infrastructure availability, and governance. Existing mechanisms might detect poisoned updates, protect gradients, audit data use, and evaluate model fairness. My research would add an infrastructure-security component that detects abnormal memory-controller, inter-socket, PCIe, or network contention and attributes it to co-located workloads.

The platform could combine these signals in a governance dashboard used by cluster operators, security engineers, and AI researchers. Policies could define acceptable interference, specify which tenants may share particular hardware paths, and determine when a job should be migrated or an attacker should be throttled. This would operationalize the paper's argument that trustworthiness requires both technical safeguards and enforceable governance, rather than relying only on voluntary good behaviour by participants.

Adaptation of my research. To better support the paper's vision, I would broaden my threat model from one specific memory-contention attack to a taxonomy of infrastructure-level threats against distributed AI. This could include contention on memory controllers, inter-socket links, PCIe, storage, and network interfaces, as well as accidental and adversarial noisy neighbours. I would then classify each threat according to detection, prevention, mitigation, and recovery.

I would also empirically probe how the survey's own defences interact with this new threat class. A concrete hypothesis worth testing is that privacy-enhancing mechanisms such as TEE-hosted local training or homomorphic encryption, which add memory-traffic and computation overhead, may in fact amplify sensitivity to co-tenant contention rather than mitigate it. A negative interaction result of this kind would be a novel contribution to the trustworthy-AI literature.

Finally, I would connect technical measurements to governance requirements. The project could define measurable service-level objectives for performance isolation and develop evidence that a cluster operator could use for auditing or policy enforcement. In the long term, this would position my work as part of trustworthy AI infrastructure: not only showing that attacks exist, but helping specify, monitor, and enforce the guarantees required for secure and reliable distributed training.

6 Evidence, originality, and GenAI-use transparency

The main sources used were the course lecture slides, the two assigned chapters of the CMU *Machine Learning in Production* book, and the two peer-reviewed papers reviewed in Section 5. The lecture slides were used to identify and interpret the selected software-engineering principles. The book chapters were used for the comparison of data scientists, software engineers, and AI engineers. The CAIN paper is a peer-reviewed conference paper, while the second source is a peer-reviewed survey in *ACM Computing Surveys*. I place the greatest trust in the peer-reviewed papers and book chapters because their claims and terminology are presented with fuller context and supporting evidence. ChatGPT was used to refine the language of the report.

References

Rimawi, Diaeddin et al. (2024). "Modeling Resilience of Collaborative AI Systems". In: *2024 IEEE/ACM 3rd International Conference on AI Engineering – Software Engineering for AI (CAIN)*, pp. 24–29.

Wei, Wenqi and Ling Liu (Feb. 2025). "Trustworthy Distributed AI Systems: Robustness, Privacy, and Governance". In: *ACM Comput. Surv.* 57.6. ISSN: 0360-0300. DOI: 10.1145/3645102. URL: https://doi.org/10.1145/3645102.